

lecture notes 16

Practical applications of the Fourier transform in data analysis.

Working with discrete data

Unfortunately, in the real world, our signals **are not continuous, and not infinitely long!** Life is short, and so is our data. This is a real shame.

For now we will assume real data

- is evenly sampled at period Δt and sampling frequency $f_s = 1/\Delta t$
- is measured for one big, uninterrupted stretch of time of length T
- has exactly N data points with $T = N\Delta t$.

Often these are not strictly true, but by studying this case we'll have an idea how to study more complicated cases in the Fourier domain.

We can model this kind of data as the “real continuous data for infinite time” $x(t)$ times a comb function (sampling) and a rectangle function (limiting time):

$$x_d(t) = \Pi(t/T) \cdot \text{III}_{\Delta t}(t) \cdot x(t)$$

Written like this, our data is zero before $t = -T/2$ and at all times after $t = T/2$. We will also label the number of data points as N , so $T = N\Delta t$.

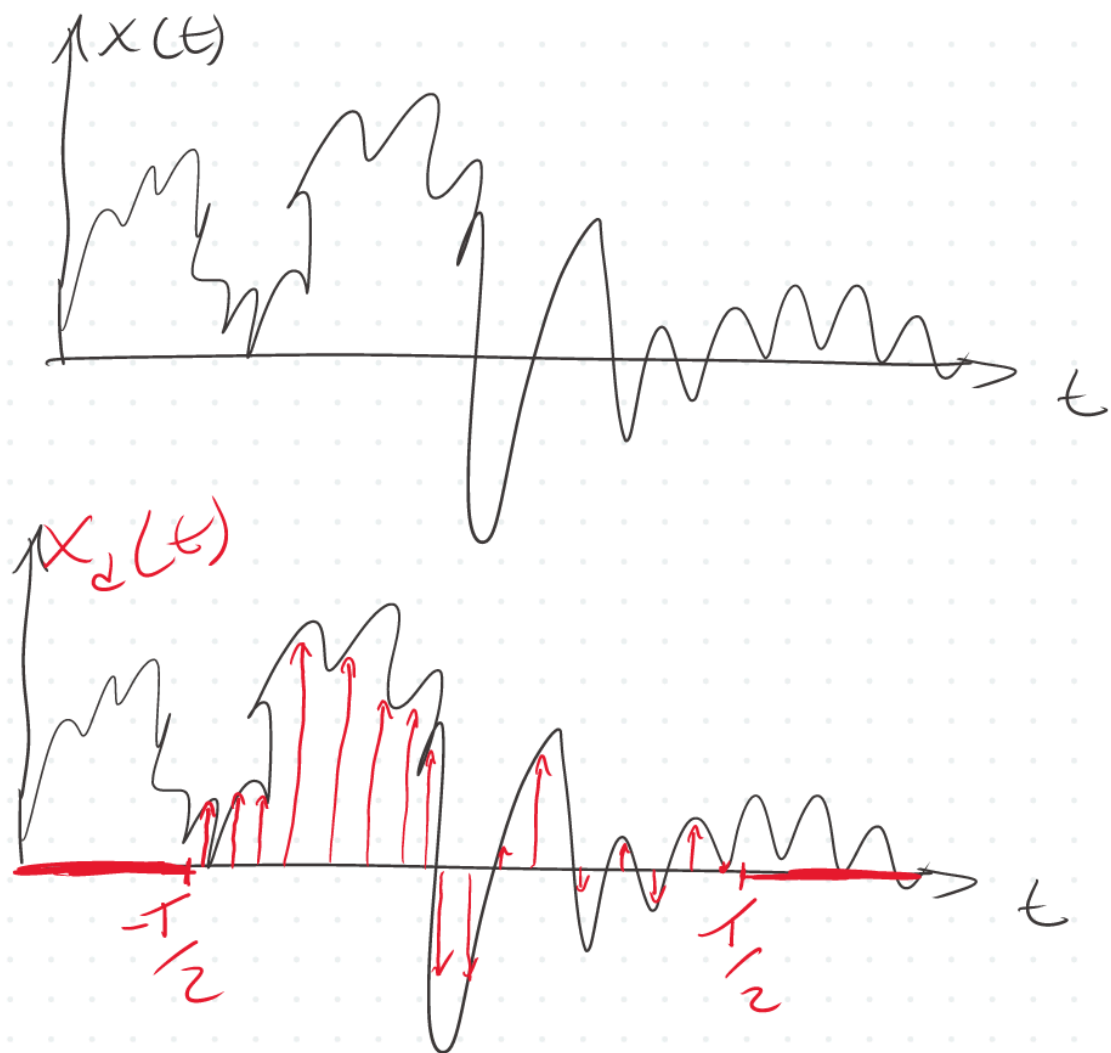


Figure 1: 1772649945338-screenshot.png

Let's study the Fourier transform in detail. This is multiplication in the time domain, so is convolution in the frequency domain.

$$\begin{aligned}
 \mathcal{F}\{\Pi(t/T) \cdot \text{III}_{\Delta t}(t) \cdot x(t)\} &= \mathcal{F}\{\Pi(t/T)\} \otimes \mathcal{F}\{\text{III}_{\Delta t}(t)\} \otimes \mathcal{F}\{x(t)\} \\
 &= T \text{sinc}(\omega T/2) \otimes \frac{1}{\Delta t} \text{III}_{1/\Delta t}(\omega) \otimes \tilde{x}(\omega) \\
 &= N \text{sinc}(\omega T/2) \otimes \text{III}_{f_s}(\omega) \otimes \tilde{x}(\omega) \\
 &= N \text{sinc}(\pi f T) \otimes \text{III}_{f_s}(f) \otimes \tilde{x}(f)
 \end{aligned}$$

First, let's just take a look at this expression overall. It's really complicated, right? **Sampling is a very destructive operation!**

Observations

Let's slowly unpack it piece by piece. This contains *all* of the effects of realistic observations on our data.

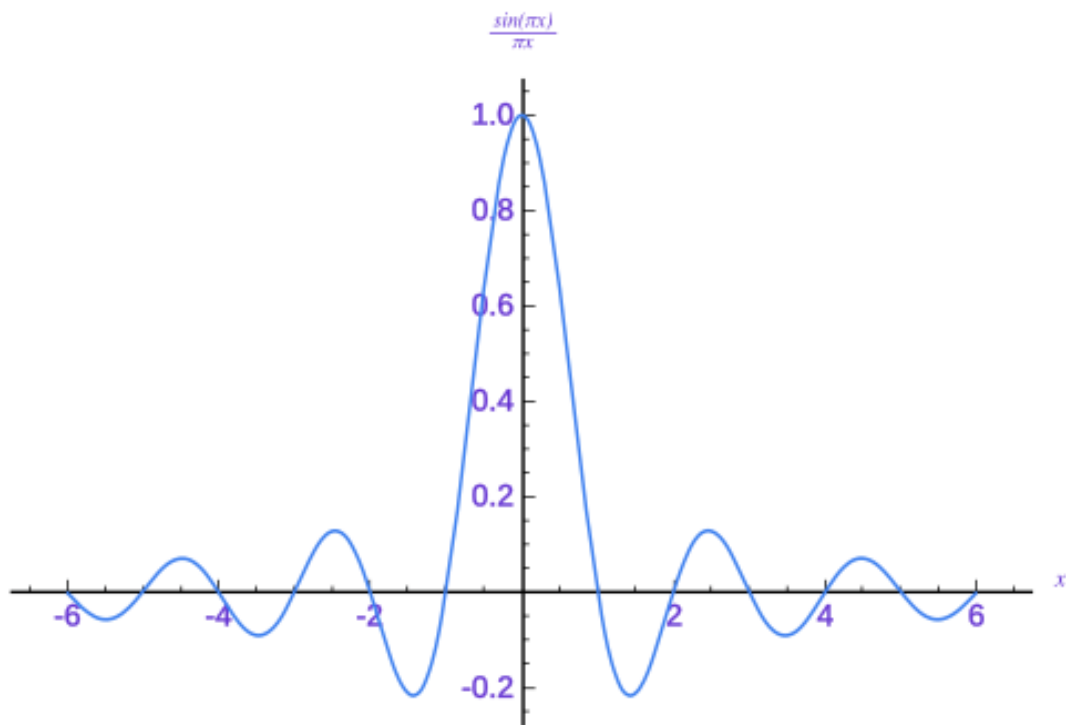


Figure 2: sinc.png

Let's start with combining the comb and the sinc. We started with a comb at Δt , but we've ended up with a comb at $f_s \equiv 1/\Delta t$. If you remember from last class, **convolution with a comb copies and shifts the other function**. (Think about the "square-ish wave" or train of gaussians examples) So a comb applied to a sinc gives us an infinite train of sinc functions

$$\begin{aligned} N \text{sinc}(\pi f T) \otimes \text{III}_{f_s}(f) &= N \text{sinc}(\pi f T) \otimes \sum_k \delta(f - k f_s) \\ &= N \sum_k \text{sinc}(\pi k f_s T) \end{aligned}$$

So there are sinc peaks at $f = 0, \pm f_s, \pm 2f_s, \dots$

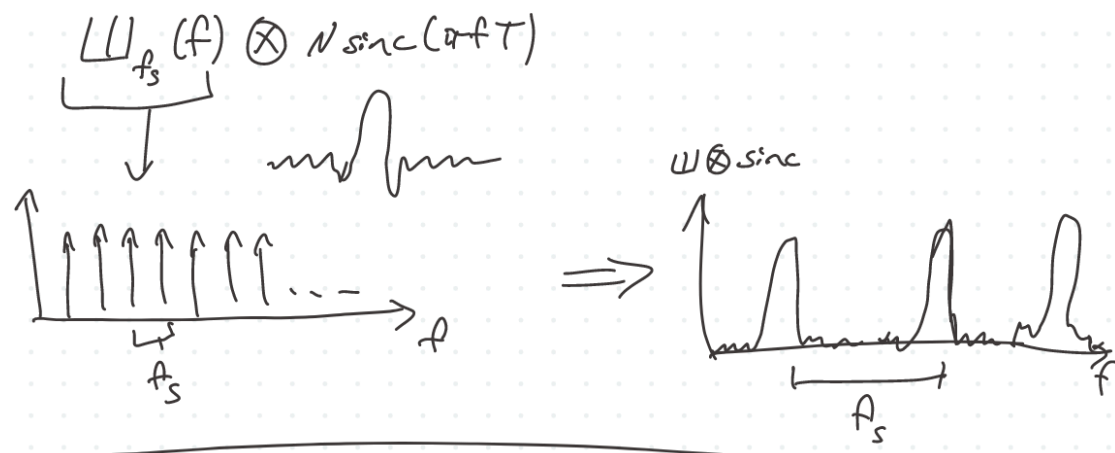


Figure 3: sinc_comb_convolution.png

And then our final Fourier transform becomes

$$\mathcal{F}\{x_d(t)\} = \tilde{x}(f) \otimes N \sum_k \text{sinc}(\pi k f_s T)$$

Convolution with a train of sincs does two things to our Fourier transform:

- We get copies of $\tilde{x}(f)$ every f_s (same idea as convolution with a comb). This means that any frequency content at f_0 will also exist at $f_0 + k f_s$. This is called **aliasing**.

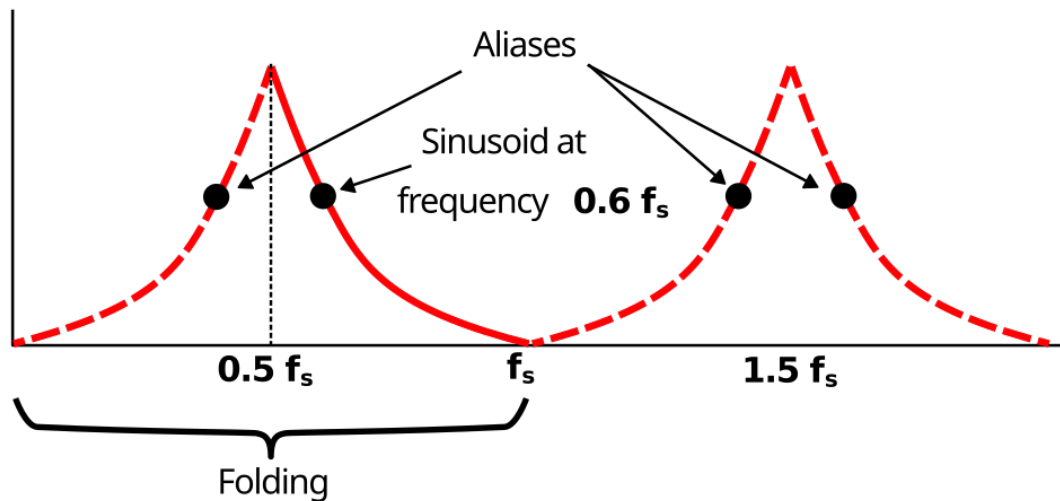


Figure 4: 060f9dc6c055c6f1c684fa1bf33f42a6.png

- This aliasing means that the frequencies we can unambiguously resolve only go up to $f_s/2$. This is called the **Nyquist frequency** and is the maximum frequency we can confidently measure. If we had any higher frequencies, they would get aliased to lower frequencies. See the animation here: https://en.wikipedia.org/wiki/Aliasing#/media/File:FFT_aliasing_600.gif

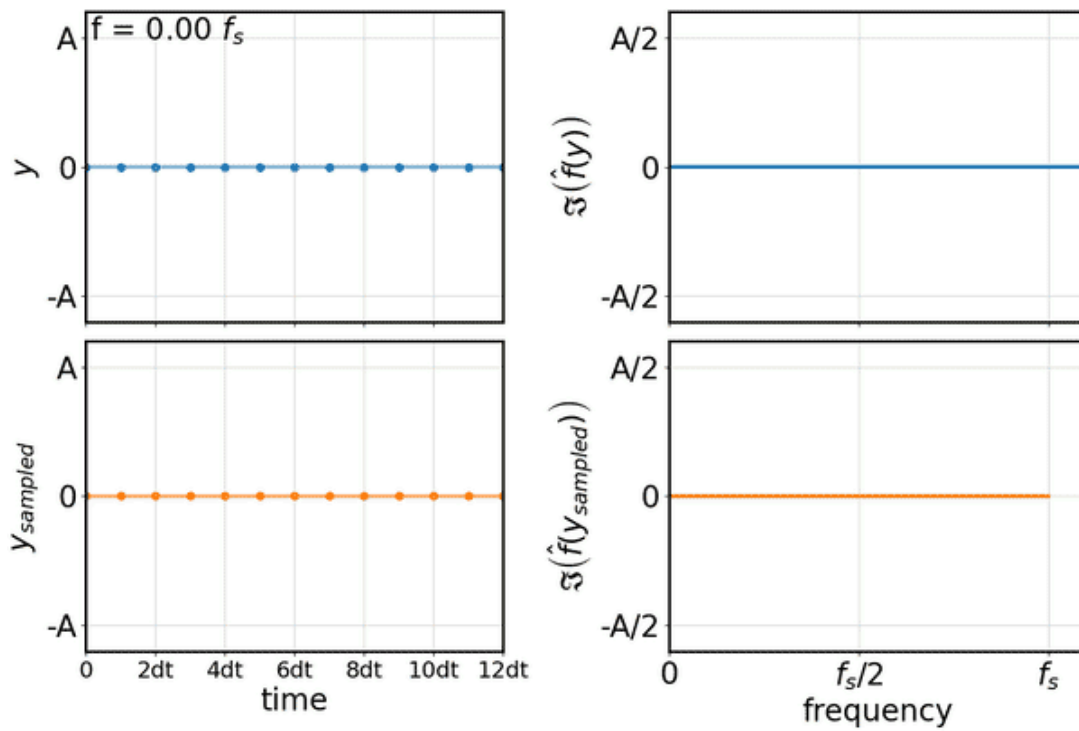


Figure 5: FFT_aliasing_600.gif

- $\tilde{x}(f)$ is *blurred* by the sinc! Imagine if $\tilde{x}(f) = \delta(f)$, then convolution with a sinc would smear out the delta function into a sinc. The width of the main lobe of the sinc is $\sim 1/T$, so frequency differences smaller than $1/T$ are blended. **This is usually taken to mean that our frequency resolution is limited at $\Delta f = 1/T$.**

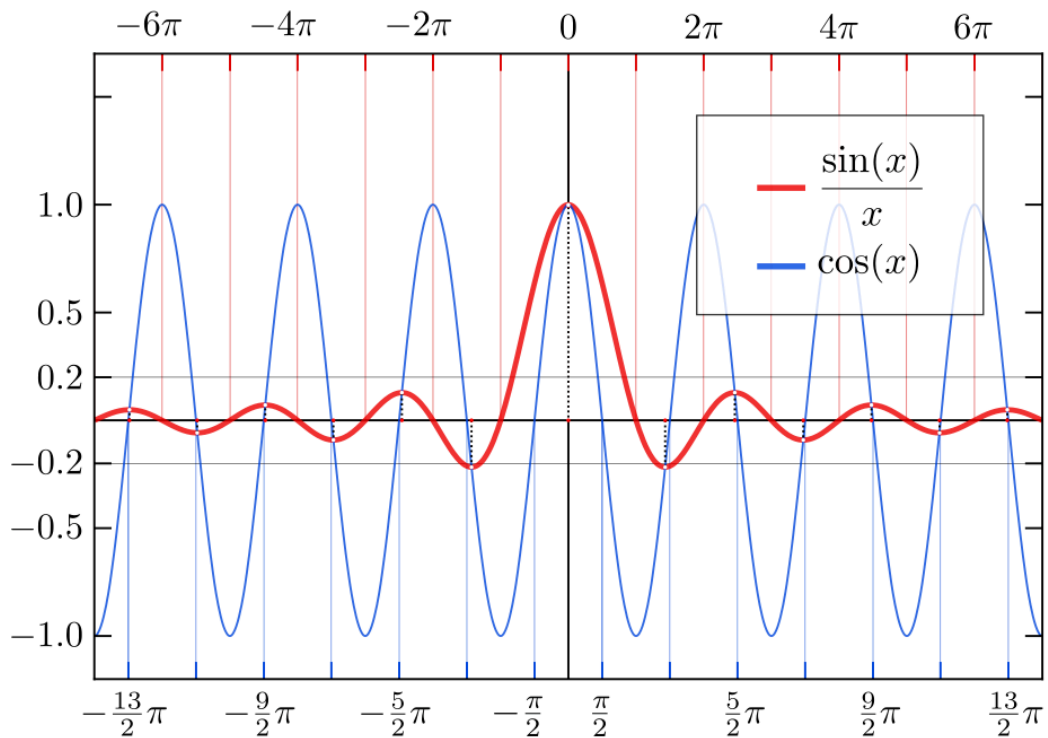


Figure 6: 5fd5f2ac5a90762cd2e4f5c460556ef4.png

In summary:

- Sampling our data gives us aliasing ($f \rightarrow f + kf_s$) and the Nyquist limit (frequencies above $f_s/2$ are ambiguous).
- The finite observation time gives us limited frequency resolution ($\Delta f = 1/T$)

Examples of this

Audio files are often sampled at 44,100 Hz. Humans can hear up to $\sim 20,000$ Hz, so this sampling rate should be able to reproduce all sounds we can hear.

Windowing

When we took our data, we chose to write the discrete data in terms of a rectangle function turning on and off

$$x_d(t) = \boxed{\Pi(t/T)} \cdot \text{III}_{\Delta t}(t) \cdot x(t)$$

We don't necessarily need to turn our data on so sharply, we could instead slowly ramp up the data. This would have the effect of changing the sinc functions to something else! This process is known as **windowing** the data, and helps stop how much single frequencies spread out. Note that the second highest peak of the sinc is about 10% of the main peak!

Unfortunately, the rectangle is the sharpest window (smallest Δf), but the others have some advantages. They can suppress how much **leakage** comes from one frequency to others.

In practice, you almost always want to use a non-rectangular window on your data to suppress leakage.

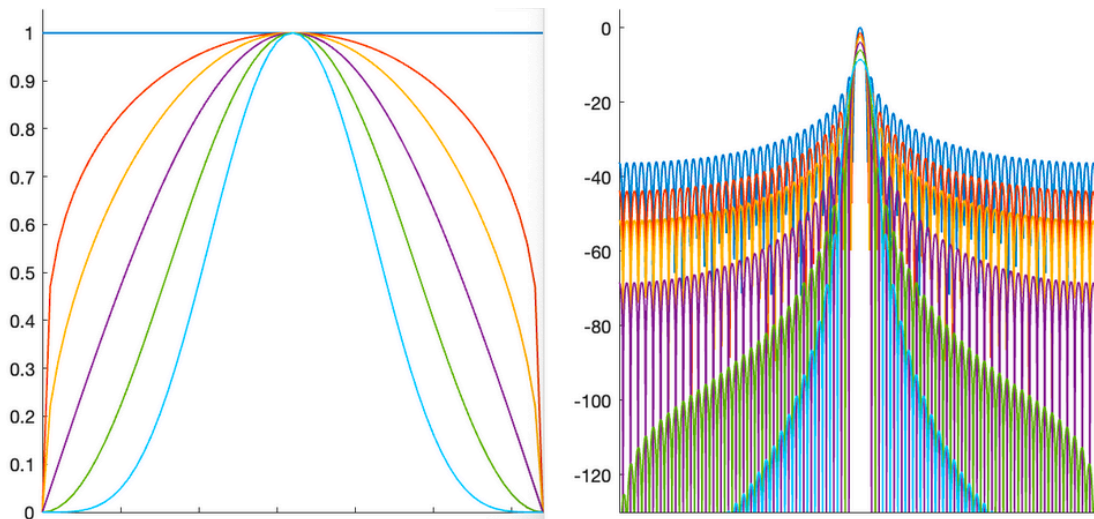


Figure 7: bae82f7bd8a6fb2f6482328c08fd5013.png

Good list here: https://en.wikipedia.org/wiki/Window_function

Discrete Fourier transform (and the FFT)

There is a discrete Fourier transform (DFT) that we use in practice with discrete data. Essentially, to go from the continuous Fourier transform to the DFT, we:

- take one copy of the aliased Fourier signal, and
- only take frequency samples every $1/T$

From our observations earlier, this is basically all the information in the Fourier transform of $x_d(t)$ anyway. (Note: in the pic below, they use P where we use T , and they use T where we use Δt)

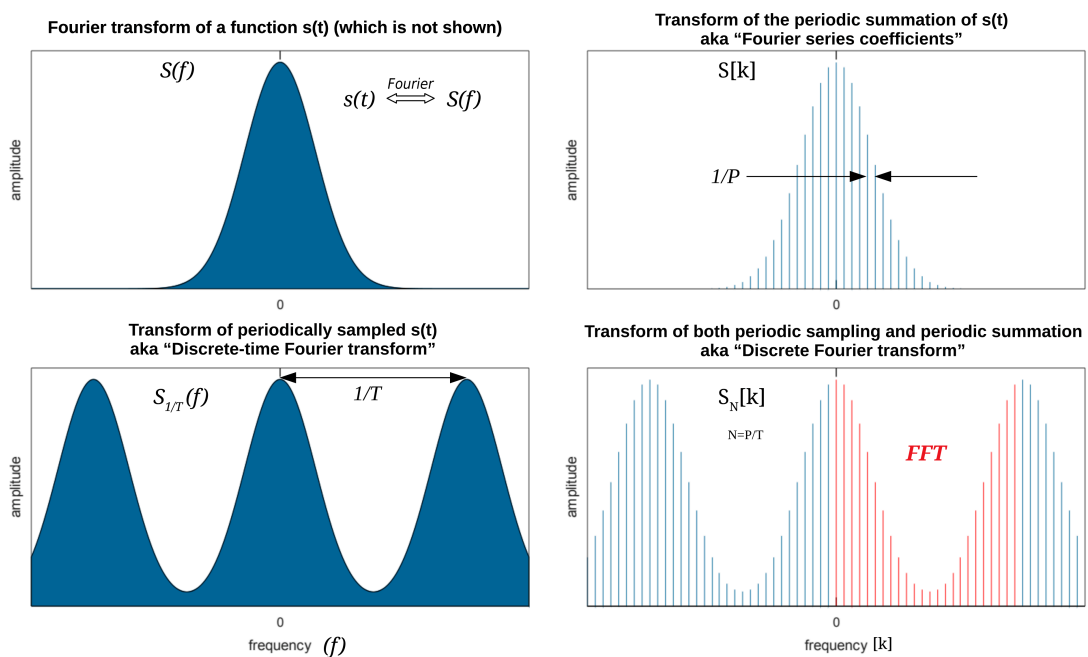


Figure 8: 1f31e4319635e131000ab23b0246f5c1.png

There is a fast algorithm, the fast Fourier transform (FFT) for computing the DFT. In practice, almost all the results we talked about last class stay true for the DFT, just like the continuous transform.

Power spectral density (PSD)

The PSD of a signal shows up often in GW data analysis. Essentially, it is just the absolute value of the FFT. We also usually scale the PSD so that it stays the same amplitude as we add more data. In the usual conventions (matches the FFT in Python's numpy):

$$\text{PSD}\{x\} \equiv |\text{FFT}\{x\}|^2 \frac{N}{2\Delta t}$$

The PSD loses some information of the signal (we only have the absolute value of the FFT, not the real and imaginary parts). But it is very useful if we just want to know how much power is in the data at a given frequency. Also, **if we are studying noise, then the PSD measures the variance of the noise at each frequency.**

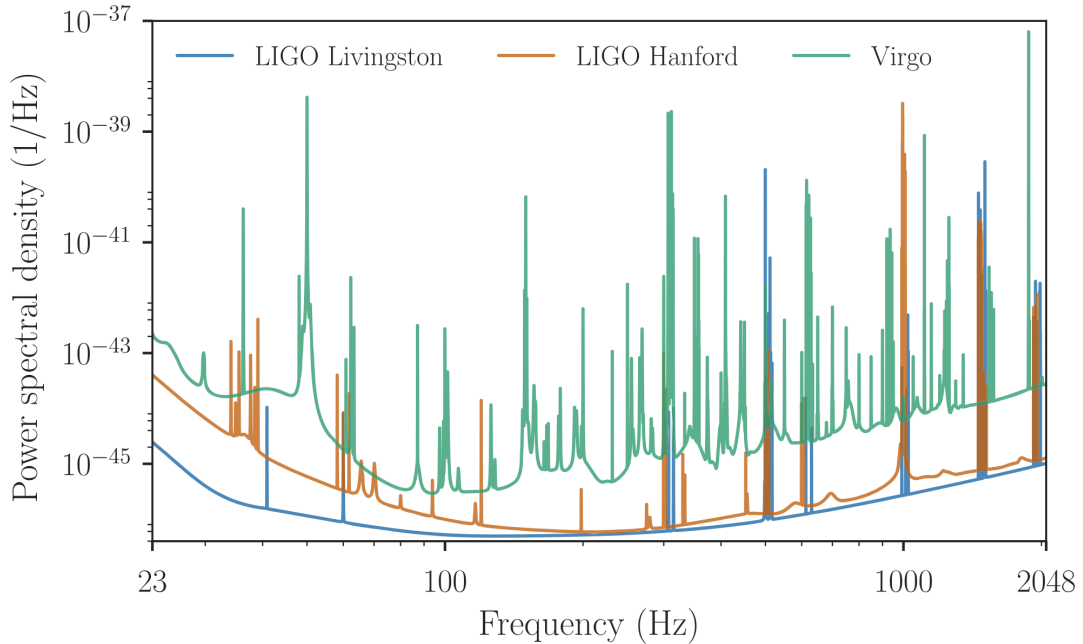


Figure 9: ligo_psd.png

Above we show the PSD of the data in the two LIGO detectors and the Virgo detector (centered around the time of a binary neutron star merger, GW170817). In this case, this is mostly the noise of the instrument. Note that the PSD is much higher towards low frequencies and high frequencies – this is typical for GW interferometers. Below we show the projected $\sqrt{\text{PSD}}$ for LISA, with a roughly similar shape:

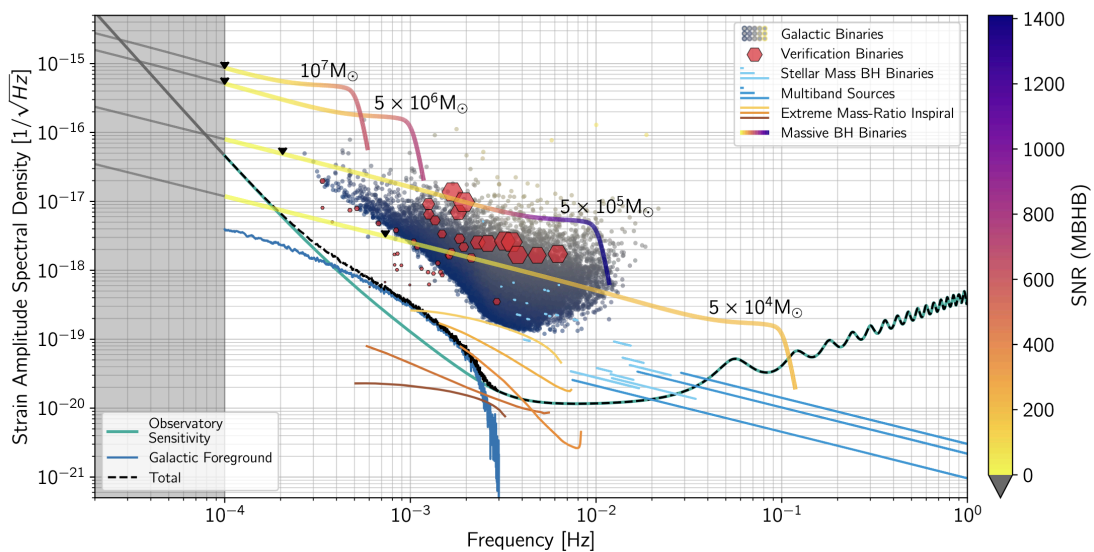


Figure 10: lisa_psd.png

The Fourier transform of stationary noise

When the PSD of some data doesn't change over time, we call it **stationary**. When data has stationary noise, we can use the diagonal covariance Gaussian likelihood we were studying last week. We'll talk more about this next week.

More topics in Fourier analysis

This is a *really deep* subject, and we could easily fill a whole semester with different topics. Ask me about these if you're curious.

- Gaps in the observations
- Tricks to beat the Fourier uncertainty principle (super-resolution)
- Image processing with FFTs
- How to remove some frequencies from the data (**filtering, anti-aliasing, low pass filter, high pass filter, ...**)
- Putting in $\delta(t)$ to measure how the system responds to any signal (**impulse response filters**)
- time-frequency methods

Generalized integral transforms

These are essentially like the Fourier transform, but instead of expressing a signal in terms of sine waves, they express it in terms of another basis! Some names to look up if you're curious:

- Laplace transform,
- G-transform,
- Mellin transform,
- Zak transform,
- Legendre transform (basis are Legendre polynomials),
- Spherical harmonics (only works for functions on the sphere)
- continuous wavelet transform.

A few of these are used in physics but are not really presented the same way as the Fourier transform. Most of them have some of the nice properties of the Fourier transform that carry over, but are better at solving some problems than others.

One example: I had a friend in grad school who “solved” a differential equation in plasma physics by inventing a new integral transform that made the equation very easy to rearrange, and writing the answer in terms of the inverse transform.

One that I have been working on a lot recently is called the **wavelet basis**, and does not use a basis of sine waves, but oscillations localized in time. These kinds of transforms let us make a **time-frequency** map of the data, to see what frequencies happen at which times. These are most useful for non-stationary data.

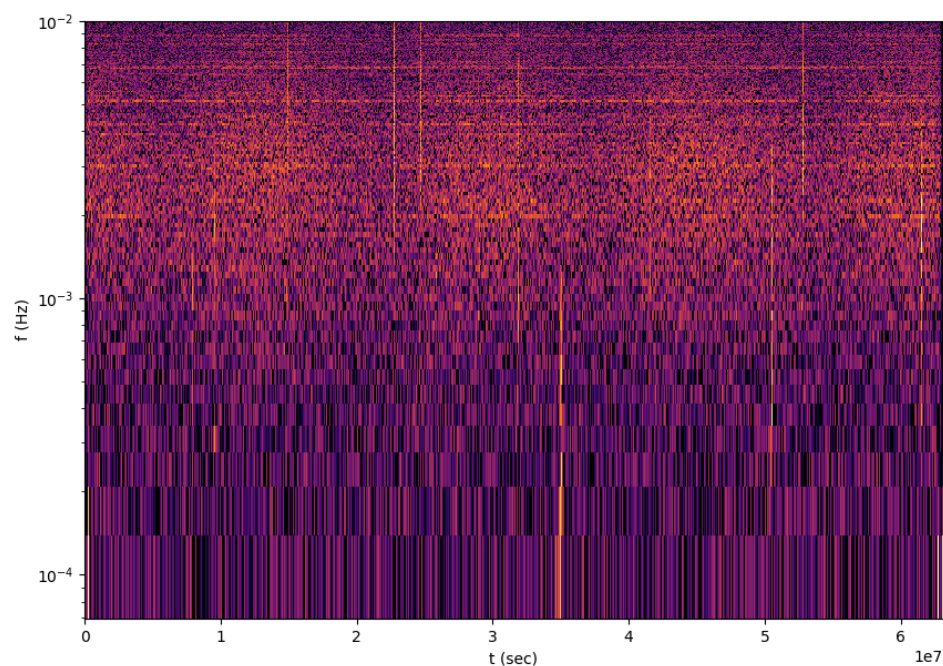


Figure 11: mojito_website2.png